Table 6.1 The primary nodes used in behavior trees

Node	Purpose	Function	Type
Selector (fallback)	This node works by selecting the first child that completes successfully. It's often called the fallback node because it will always fall back to the last successful node that executed.	The node calls its children in sequence and stops executing when the first child succeeds. When a child node succeeds, it will return success; if no nodes succeed, it returns failure.	Composite
Sequence	This node executes all of its children in sequence until one node fails or they all complete successfully.	The node calls each of its children in sequence regardless of whether they fail or succeed. If all children succeed, it returns success, and failure if just one child fails.	Composite
Condition	Behavior trees don't use Boolean logic but rather success or failure as a means of control. The condition returns success if the condition is true and false otherwise.	The node returns success or failure based on a condition.	Task
Action	This is where the action happens.	The node executes and returns success if successful or returns failure otherwise.	Task
Decorator	They work by controlling the execution of child nodes. They are often referred to as conditionals because they can determine whether a node is worth executing or safe to execute.	The node controls execution of the child nodes. Decorators can operate as control barrier functions to block or prevent unwanted behaviors.	Decorator
Parallel	This node executes all of its nodes in parallel. Success or failure is controlled by a threshold of the number of children needed to succeed to return success.	The node executes all of its child nodes in sequence regardless of the status of the nodes.	Composite

The primary nodes in table 6.1 can provide enough functionality to handle numerous use cases. However, understanding behavior trees initially can be daunting. You won't appreciate their underlying complexity until you start using them. Before we build some simple trees, we want to look at execution in more detail in the next section.